

Sintaxis de Java para live coding

Un repaso Java-first, compatible con Java 17+, para recuperar soltura antes de programar funciones, modelar datos y resolver ejercicios en directo.

Objetivo: reconocer y escribir la sintaxis sin depender de memoria perfecta. La práctica posterior mezclará ejercicios y no seguirá necesariamente este orden.

Base	Enfoque	Uso
Java 17+	Sintaxis estable	Compatible con tu JDK 25
Sin Spring	Funciones y objetos	Live coding rápido
Main + casos	Ejecutar pronto	Detectar errores en directo

Preparado para Daniel - entrevista backend / Dribba

Cómo estudiar esta guía

Primera vuelta: lee y ejecuta mentalmente los ejemplos. Segunda vuelta: escribe sin copiar los bloques marcados como esenciales. Tercera vuelta: usa la guía solo para consultar dudas concretas.

Índice

Bloque	Contenido
1-4	Programa mínimo, variables, operadores, control de flujo
5-8	Métodos, arrays, strings, listas y colecciones
9-12	Map/Set, ordenación, clases, records y enums
13-16	Interfaces, herencia, genéricos y excepciones
17-20	Optional, lambdas, streams, fechas y tiempo
21-24	Igualdad, mutabilidad, entrada/salida y testing
25-27	Complejidad, patrones de live coding y chuleta final

Regla de entrevista

Primero aclara el contrato. Después resuelve de forma clara. Ejecuta. Solo entonces optimiza o extrae abstracciones.

1. Programa mínimo y estructura

Todo archivo Java suele declarar un paquete, imports y uno o más tipos. El nombre de una clase pública debe coincidir con el archivo.

```
package practice;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> values = List.of(1, 2, 3);
        int result = sum(values);
        System.out.println(result);
    }

    public static int sum(List<Integer> values) {
        int total = 0;
        for (int value : values) {
            total += value;
        }
        return total;
    }
}
```

- **public**: accesible desde cualquier clase.
- **static**: pertenece a la clase; no requiere crear un objeto.
- **void**: no devuelve valor.
- **main**: punto de entrada habitual para ejecutar rápidamente.

Comentarios

```
// Una línea
/* Varias líneas */
/** Javadoc para documentar tipos y métodos públicos. */
```

2. Variables, tipos y literales

Tipo	Ejemplo	Idea
int	int count = 10;	Entero de 32 bits
long	long total = 10L;	Entero de 64 bits
double	double price = 2.5;	Decimal aproximado
boolean	boolean open = true;	true / false
char	char initial = 'D';	Un carácter
String	String name = "Daniel";	Objeto, no primitivo

```
int age = 25;
var city = "Barcelona"; // El compilador infiere String
final int MAX_SIZE = 100; // No se puede reasignar
```

```
Integer boxed = age;      // boxing automático
int primitive = boxed;    // unboxing automático
```

Primitivos frente a wrappers

Las colecciones usan objetos: `List<Integer>`, no `List<int>`. Los wrappers pueden ser null; los primitivos no.

Riesgo: hacer unboxing de un Integer null produce NullPointerException.

3. Operadores y conversiones

```
int a = 7;
int b = 3;

int sum = a + b;           // 10
int quotient = a / b;      // 2: división entera
int remainder = a % b;     // 1
double exact = (double) a / b; // 2.333...

boolean valid = a > 0 && b > 0;
boolean either = a == 7 || b == 7;
boolean opposite = !valid;

int max = a >= b ? a : b; // operador ternario
```

- Comparación: `==` `!=` `<` `<=` `>` `>=`.
- Lógicos con cortocircuito: `&&` y `||`.
- Asignación abreviada: `+=` `-=` `*=` `/=`.
- Incremento: `i++`, decremento: `i--`.

Precedencia

Cuando haya mezcla de operadores, usa paréntesis para que la intención sea evidente. En live coding se valora la claridad más que demostrar que recuerdas toda la tabla de precedencia.

4. Condicionales y bucles

```
if (score >= 90) {
    grade = "A";
} else if (score >= 70) {
    grade = "B";
} else {
    grade = "C";
}

for (int i = 0; i < values.size(); i++) {
    System.out.println(values.get(i));
}

for (String value : values) {
    System.out.println(value);
}

while (condition) {
    // repetir mientras sea true
}
```

break, continue y switch

```
for (int value : values) {
    if (value < 0) continue;
    if (value == target) break;
}

String label = switch (status) {
    case PENDING -> "Pendiente";
    case PAID -> "Pagado";
}
```

```
    case CANCELLED -> "Cancelado";  
};
```

En un bucle por índice, la última posición válida es size - 1. El error clásico es usar i <= size.

5. Métodos, parámetros y retorno

```
public static int add(int left, int right) {  
    return left + right;  
}  
  
public static void printMessage(String message) {  
    System.out.println(message);  
}  
  
public static int sum(int... values) { // varargs  
    int total = 0;  
    for (int value : values) total += value;  
    return total;  
}
```

Sobrecarga

```
int parse(String value) { ... }  
int parse(String value, int defaultValue) { ... }
```

La sobrecarga usa el mismo nombre con parámetros diferentes. No se puede sobrecargar únicamente cambiando el tipo de retorno.

Paso de parámetros

Java siempre pasa por valor. Para objetos, el valor copiado es la referencia. Puedes modificar el objeto apuntado, pero reasignar el parámetro no cambia la variable del llamador.

```
void rename(User user) { user.setName("Ana"); } // modifica objeto  
void replace(User user) { user = new User(); }    // no reemplaza variable externa
```

6. Arrays

```
int[] numbers = {3, 1, 4};  
int[] empty = new int[5]; // cinco ceros  
  
int first = numbers[0];  
numbers[1] = 8;  
int length = numbers.length;  
  
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}  
  
for (int number : numbers) {  
    System.out.println(number);  
}
```

Utilidades

```
import java.util.Arrays;  
  
Arrays.sort(numbers);  
int position = Arrays.binarySearch(numbers, 4); // array ordenado  
int[] copy = Arrays.copyOf(numbers, numbers.length);  
boolean same = Arrays.equals(numbers, copy);  
System.out.println(Arrays.toString(numbers));
```

- Tamaño fijo después de crearlo.
- Acceso por índice $O(1)$.
- Los arrays de objetos se inicializan con null.
- Un array bidimensional: `int[][] matrix = new int[3][4];`.

7. String y StringBuilder

```
String text = "  Hola Java  ";
String clean = text.trim();

int length = clean.length();
char first = clean.charAt(0);
String part = clean.substring(0, 4);
boolean contains = clean.contains("Java");
boolean starts = clean.startsWith("Hola");
String lower = clean.toLowerCase();
String replaced = clean.replace("Java", "Daniel");
String[] words = clean.split("\\s+");
```

Comparar strings

```
name.equals("Daniel");
"Daniel".equals(name); // seguro si name puede ser null
name.equalsIgnoreCase("daniel");

// Evitar para contenido:
name == "Daniel";
```

Construcción eficiente

```
StringBuilder builder = new StringBuilder();
for (String word : words) {
    builder.append(word).append(' ');
}
String result = builder.toString().trim();
```

Método	Ejemplo -> resultado	Qué hace
<code>trim()</code>	<code>" hola ".trim()</code> -> <code>"hola"</code>	Quita espacios iniciales y finales
<code>length()</code>	<code>"java".length()</code> -> 4	Número de caracteres
<code>charAt(1)</code>	<code>"java".charAt(1)</code> -> 'a'	Carácter de una posición
<code>substring(1,3)</code>	<code>"java".substring(1,3)</code> -> <code>"av"</code>	Extrae desde inicio incluido hasta fin excluido
<code>contains()</code>	<code>"hola java".contains("java")</code> -> true	Comprueba si contiene texto
<code>startsWith()</code>	<code>"java".startsWith("ja")</code> -> true	Comprueba el comienzo
<code>toLowerCase()</code>	<code>"JAVA".toLowerCase()</code> -> <code>"java"</code>	Convierte a minúsculas
<code>replace()</code>	<code>"casa".replace("a","o")</code> -> <code>"coso"</code>	Sustituye coincidencias
<code>split(",")</code>	<code>"a,b,c".split(",")</code> -> <code>[a,b,c]</code>	Divide en un array
<code>isBlank()</code>	<code>" ".isBlank()</code> -> true	Vacío o formado solo por espacios

String es inmutable. Concatenar repetidamente dentro de un bucle puede crear muchos objetos; usa StringBuilder.

8. List y sus implementaciones

```
List<String> names = new ArrayList<>();
names.add("Ana");
names.add("Daniel");
names.add(1, "Marta");

String first = names.get(0);
names.set(0, "Alex");
names.remove("Marta");
names.remove(0);
boolean found = names.contains("Daniel");
int size = names.size();
```

Operación	Ejemplo	Resultado / efecto
add(x)	[A].add(B)	[A, B]
get(0)	[A, B].get(0)	A
set(0,X)	[A, B].set(0,X)	[X, B]
remove(0)	[A, B].remove(0)	[B]
contains(B)	[A, B].contains(B)	true
size()	[A, B].size()	2
isEmpty()	{}.isEmpty()	true

Crear listas

```
List<Integer> fixed = List.of(1, 2, 3); // immutable
List<Integer> mutable = new ArrayList<>(fixed);
List<Integer> copy = new ArrayList<>(mutable);
```

Tipo	Fortaleza	Debilidad
ArrayList	Acceso por índice; uso general	Insertar al principio desplaza elementos
LinkedList	Añadir/quitar en extremos	Acceso por índice lento; rara vez la mejor opción

List.of(...) no permite add, remove ni null. Si debes modificarla, crea new ArrayList<>(lista).

9. Set: unicidad y pertenencia

```
Set<String> seen = new HashSet<>();

for (String value : values) {
    if (!seen.add(value)) {
        System.out.println("Duplicado: " + value);
    }
}

boolean exists = seen.contains("Daniel");
seen.remove("Daniel");
```

Método	Resultado
add(x)	Añade; devuelve false si ya existía

Método	Resultado
contains(x)	Indica si pertenece al conjunto
remove(x)	Elimina y devuelve si existía
size()	Cantidad de elementos únicos

Implementación	Orden
HashSet	Sin orden garantizado; operaciones promedio $O(1)$
LinkedHashSet	Conserva orden de inserción
TreeSet	Ordenado; operaciones $O(\log n)$

Operaciones entre conjuntos

```
Set<Integer> intersection = new HashSet<>(a);
intersection.retainAll(b);
```

```
Set<Integer> union = new HashSet<>(a);
union.addAll(b);
```

```
Set<Integer> difference = new HashSet<>(a);
difference.removeAll(b);
```

10. Map: claves, valores y conteos

```
Map<String, Integer> counts = new HashMap<>();
```

```
for (String word : words) {
    counts.put(word, counts.getOrDefault(word, 0) + 1);
}
```

```
int count = counts.getOrDefault("java", 0);
boolean hasKey = counts.containsKey("java");
```

```
for (Map.Entry<String, Integer> entry : counts.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

Método	Ejemplo / significado
put(k,v)	Inserta o reemplaza el valor de la clave
get(k)	Devuelve el valor o null si no existe
getOrDefault(k,0)	Devuelve el valor; si falta, devuelve 0
containsKey(k)	Comprueba si existe la clave
keySet()	Conjunto de claves
values()	Colección de valores
entrySet()	Pares clave-valor para recorrer

Métodos útiles

```
map.putIfAbsent(key, value);
map.computeIfAbsent(key, ignored -> new ArrayList<>()).add(item);
map.merge(key, 1, Integer::sum);
```



```
map.remove(key);
```

Método	Qué hace	Ejemplo breve
getOrDefault	Lee o devuelve un valor alternativo	counts.getOrDefault(word, 0)
putIfAbsent	Inserta solo si falta la clave	map.putIfAbsent(id, user)
computeIfAbsent	Crea el valor si falta y lo devuelve	groups.computeIfAbsent(k, x -> new ArrayList<>())
merge	Inserta o combina con el valor anterior	counts.merge(word, 1, Integer::sum)

Implementación	Uso
HashMap	General, sin orden garantizado
LinkedHashMap	Mantener orden de inserción
TreeMap	Claves ordenadas

11. Ordenación y Comparator

```
List<Integer> values = new ArrayList<>(List.of(3, 1, 2));
Collections.sort(values);
values.sort(Comparator.naturalOrder());
values.sort(Comparator.reverseOrder());
```

```
names.sort(Comparator.comparingInt(String::length)
              .thenComparing(Comparator.naturalOrder()));
```

Comparator	Qué hace	Ejemplo breve
naturalOrder	Orden natural ascendente	1,2,3 / A,B,C
reverseOrder	Orden natural descendente	3,2,1 / C,B,A
comparing	Extrae una clave Comparable	comparing(Order::createdAt)
comparingInt	Extrae una clave int sin boxing	comparingInt(String::length)
reversed	Invierte el comparador anterior	comparing(Product::price).reversed()
thenComparing	Desempata con otro criterio	...thenComparing(Product::name)

Ordenar objetos por varios campos

```
orders.sort(
    Comparator.comparing(Order::createdAt)
               .reversed()
               .thenComparing(Order::id)
);
```

Máximo y mínimo

```
Comparator<Order> byTotal = Comparator.comparing(Order::total);
Order cheapest = orders.stream().min(byTotal).orElseThrow();
Order mostExpensive = orders.stream().max(byTotal).orElseThrow();
```

list.sort(...) modifica la lista. **stream().sorted(...).toList()** crea un resultado ordenado sin modificar por sí solo la lista original.

12. Clases, constructores y encapsulación

```
public class Order {
    private final long id;
    private OrderStatus status;

    public Order(long id) {
        if (id <= 0) throw new IllegalArgumentException("Invalid id");
        this.id = id;
        this.status = OrderStatus.PENDING;
    }

    public long getId() {
        return id;
    }

    public OrderStatus getStatus() {
        return status;
    }

    public void cancel() {
        if (status != OrderStatus.PENDING) {
            throw new IllegalStateException("Order cannot be cancelled");
        }
        status = OrderStatus.CANCELLED;
    }
}
```

- **private** oculta detalles internos.
- **final** evita reasignar el campo después de construir el objeto.
- **this** se refiere al objeto actual.
- Coloca invariantes cerca del estado que protegen.

13. Records y enums

Record

Un record representa datos inmutables y genera constructor, accesores, equals, hashCode y toString.

```
public record TimeRange(LocalTime start, LocalTime end) {
    public TimeRange {
        if (start == null || end == null) {
            throw new IllegalArgumentException("Times are required");
        }
    }
}

TimeRange range = new TimeRange(LocalTime.of(9, 0), LocalTime.of(14, 0));
LocalTime start = range.start();
```

Enum

```
public enum OrderStatus {
    PENDING,
    PAID,
    CANCELLED
}

if (order.status() == OrderStatus.PENDING) { ... }
```

Para datos pequeños del enunciado, record suele ser mejor que escribir una clase llena de getters. Para estado y comportamiento mutable, una clase puede encajar mejor.

14. Interfaces, polimorfismo y composición

```
public interface PaymentProvider {
    PaymentResult pay(PaymentRequest request);
}

public final class StripePaymentProvider implements PaymentProvider {
    @Override
    public PaymentResult pay(PaymentRequest request) {
        // adaptar petición y respuesta de Stripe
        return PaymentResult.success();
    }
}

public final class OrderService {
    private final PaymentProvider paymentProvider;

    public OrderService(PaymentProvider paymentProvider) {
        this.paymentProvider = paymentProvider;
    }
}
```

- Una interfaz define el contrato del que quieres depender.
- Una clase usa implements; una interfaz puede usar extends.
- Una clase puede implementar varias interfaces.
- No crees interfaces automáticamente: deben aportar una frontera o sustitución útil.

Composición

OrderService tiene un PaymentProvider. Preferimos esta relación a herencias artificiales.

15. Herencia, clases abstractas y modificadores

```
public abstract class Shape {
    public abstract double area();

    public String description() {
        return "Area: " + area();
    }
}

public final class Circle extends Shape {
    private final double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

Modificador	Acceso
public	Desde cualquier paquete
protected	Mismo paquete y subclases
sin modificador	Solo mismo paquete
private	Solo dentro de la clase

Usa herencia cuando exista una relación 'es un' estable y se respete el contrato. En caso de duda, composición.

16. Genéricos

```
public static <T> T first(List<T> values) {
    if (values.isEmpty()) {
        throw new IllegalArgumentException("Empty list");
    }
    return values.get(0);
}
```

```
String firstName = first(List.of("Ana", "Daniel"));
Integer firstNumber = first(List.of(10, 20));
```

Límites y comodines

```
public static double sum(List<? extends Number> values) {
    double total = 0;
    for (Number value : values) total += value.doubleValue();
    return total;
}
```

```
public static void addDefaults(List<? super Integer> target) {
    target.add(0);
}
```

Regla PECS: **P**roducer **E**xtends, **C**onsumer **S**uper. Si lees valores T, suele aparecer ? extends T; si introduces valores T, ? super T.

Para la mayoría de live codings bastará con List, Map y métodos genéricos sencillos.

17. Excepciones y validación

```
public int parsePositive(String value) {
    try {
        int number = Integer.parseInt(value);
        if (number <= 0) {
            throw new IllegalArgumentException("Must be positive");
        }
        return number;
    } catch (NumberFormatException exception) {
        throw new IllegalArgumentException("Invalid number: " + value, exception);
    }
}
```

Checked y unchecked

Tipo	Ejemplo	Tratamiento
Checked	IOException	Capturar o declarar throws
Unchecked	IllegalArgumentException	No obliga a capturar

```
try {
    riskyOperation();
} catch (SpecificException exception) {
    // manejar caso conocido
} finally {
    // se ejecuta al terminar, haya o no error
}
```

18. Optional y ausencia de valor

```
Optional<User> user = repository.findById(id);

String name = user.map(User::name)
                  .orElse("Unknown");

User required = user.orElseThrow(
    () -> new UserNotFoundException(id)
);

user.ifPresent(value -> System.out.println(value.name()));
```

Método	Qué hace	Ejemplo breve
map	Transforma el valor si existe	user.map(User::name)
orElse	Valor alternativo ya calculado	name.orElse("Unknown")
orElseGet	Calcula el alternativo solo si falta	value.orElseGet(this::loadDefault)
orElseThrow	Devuelve o lanza una excepción	user.orElseThrow()
ifPresent	Ejecuta una acción si existe	user.ifPresent(this::print)

- `Optional.empty()` representa ausencia.
- Evita llamar a `get()` sin comprobar.
- No suele usarse como campo de entidad ni parámetro de todos los métodos.
- Para colecciones sin resultados, normalmente devuelve una colección vacía, no `Optional` de lista.

orElse frente a orElseGet

```
value.orElse(expensiveDefault()); // evalúa siempre
value.orElseGet(() -> expensiveDefault()); // evalúa solo si está vacío
```

19. Lambdas y referencias a métodos

```
Predicate<String> longEnough = text -> text.length() >= 5;
Function<String, Integer> length = String::length;
Consumer<String> printer = System.out::println;
Supplier<List<String>> factory = ArrayList::new;
```

```
names.removeIf(String::isBlank);
names.forEach(System.out::println);
```

Forma	Significado	Ejemplo
x -> ...	Lambda con un parámetro	x -> x * 2
(a,b) -> ...	Lambda con dos parámetros	(a,b) -> a + b
Clase::metodo	Método de instancia sobre el argumento	String::trim
objeto::metodo	Método de un objeto concreto	System.out::println
Clase::new	Referencia a constructor	ArrayList::new

Interfaz funcional	Firma mental
Predicate<T>	T -> boolean
Function<T,R>	T -> R
Consumer<T>	T -> void
Supplier<T>	() -> T
Comparator<T>	(T, T) -> int

Una interfaz funcional tiene un único método abstracto. Las lambdas pueden capturar variables locales efectivamente finales.

20. Streams

```
List<String> result = names.stream()
    .filter(name -> !name.isBlank())
    .map(String::trim)
    .map(String::toUpperCase)
    .distinct()
    .sorted()
    .toList();
```

```
int total = numbers.stream()
    .mapToInt(Integer::intValue)
    .sum();
```

Operación	Antes -> después
filter	[1,2,3,4] -> pares -> [2,4]
map	[a,bb] -> longitudes -> [1,2]
mapToInt	[1,2,3] -> IntStream; permite sum()
distinct	[1,1,2] -> [1,2]
sorted	[3,1,2] -> [1,2,3]
limit(2)	[1,2,3] -> [1,2]
skip(2)	[1,2,3,4] -> [3,4]
count	[a,b,c] -> 3
findFirst	[a,b] -> Optional[a]

Terminal	Resultado mental	Ejemplo breve
anyMatch	¿Alguno cumple?	values.stream().anyMatch(x -> x < 0)
allMatch	¿Todos cumplen?	values.stream().allMatch(x -> x > 0)
noneMatch	¿Ninguno cumple?	values.stream().noneMatch(String::isBlank)
max / min	Optional con extremo	stream.max(Comparator.naturalOrder())
reduce	Combina en un valor	stream.reduce(0, Integer::sum)
toList	Crea lista resultado	stream.filter(...).toList()

Terminal	Resultado mental	Ejemplo breve
collect	Acumulación configurable	collect(groupingBy(Order::userId))

Agrupar y contar

```
Map<String, Long> counts = words.stream()
    .collect(Collectors.groupingBy(
        word -> word,
        Collectors.counting()
    ));

Map<String, List<Order>> byUser = orders.stream()
    .collect(Collectors.groupingBy(Order::userId));
```

- Intermedias: devuelven otro stream y son lazy; por ejemplo filter, map, sorted y distinct.
- Terminales: producen el resultado y consumen el stream; por ejemplo toList, count, reduce y findFirst.
- stream() no modifica por sí solo la colección original.
- list.sort() modifica la lista; stream().sorted().toList() devuelve otra lista.
- Un stream se consume una sola vez.
- Si el bucle es más claro, usa el bucle. Streams no son obligatorios.

21. Fechas, horas e intervalos

```
LocalDate date = LocalDate.of(2026, 7, 11);
LocalTime opening = LocalTime.of(9, 0);
LocalDateTime moment = LocalDateTime.of(date, opening);
DayOfWeek day = date.getDayOfWeek();

boolean open = !time.isBefore(opening)
    && time.isBefore(closing); // [opening, closing)

LocalTime parsed = LocalTime.parse("17:30");
```

Tipo / método	Ejemplo	Qué representa
LocalDate	2026-07-11	Fecha sin hora ni zona
LocalTime	17:30	Hora sin fecha ni zona
LocalDateTime	2026-07-11T17:30	Fecha y hora sin zona
isBefore	09:00.isBefore(10:00) -> true	Comparación anterior
isAfter	18:00.isAfter(10:00) -> true	Comparación posterior
plusHours(2)	09:00 -> 11:00	Añade horas sin modificar el original

Zonas horarias

```
ZoneId madrid = ZoneId.of("Europe/Madrid");
ZonedDateTime now = ZonedDateTime.now(madrid);
Instant instant = now.toInstant();
```

Duraciones

```
Duration duration = Duration.between(opening, closing);
long minutes = duration.toMinutes();
```

Para horarios de apertura, aclara intervalos que cruzan medianoche y si la hora de cierre está incluida. Evita `java.util.Date` para código nuevo.

22. equals, hashCode y orden natural

```
if (Objects.equals(left, right)) { ... } // tolera null

public final class User {
    private final long id;

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof User user)) return false;
        return id == user.id;
    }

    @Override
    public int hashCode() {
        return Long.hashCode(id);
    }
}
```

- Si dos objetos son equals, deben producir el mismo hashCode.
- HashMap y HashSet dependen de este contrato.
- Records generan equals/hashCode basados en sus componentes.
- Comparable define orden natural; Comparator define órdenes externos.

No modifiques campos usados en hashCode mientras el objeto sea clave de HashMap o elemento de HashSet.

23. Mutabilidad, copias y null

```
List<String> original = new ArrayList<>(List.of("A", "B"));
List<String> alias = original; // mismo objeto
List<String> shallowCopy = new ArrayList<>(original); // nueva lista

List<String> readOnly = Collections.unmodifiableList(original);
List<String> immutable = List.copyOf(original);
```

Copia superficial

Copiar una lista crea otra estructura, pero los objetos internos siguen siendo las mismas referencias. Si esos objetos son mutables, los cambios internos se comparten.

Gestionar null

```
Objects.requireNonNull(input, "input is required");

if (value == null) {
    return defaultValue;
}
```

En la entrevista pregunta si null es una entrada válida. No añadas una política arbitraria sin indicarlo.

24. Entrada/salida rápida y parseo

Scanner: sencillo

```
Scanner scanner = new Scanner(System.in);
int number = scanner.nextInt();
scanner.nextLine(); // consumir fin de línea pendiente
String line = scanner.nextLine();
```

BufferedReader: más control

```
BufferedReader reader = new BufferedReader(
    new InputStreamReader(System.in)
);

String line = reader.readLine();
String[] parts = line.split("\\s+");
int value = Integer.parseInt(parts[0]);
```

Imprimir colecciones

```
System.out.println(result);
System.out.printf("Expected=%s, actual=%s%n", expected, actual);
```

Si el PDF ya define llamadas a una función, no construyas un parser completo. Utiliza Main para crear entradas directamente, salvo que pidan leer stdin o un archivo.

25. Testing mínimo en directo

```
public static void main(String[] args) {
    check("normal", 6, sumEven(List.of(1, 2, 4)));
    check("empty", 0, sumEven(List.of()));
    check("negatives", -6, sumEven(List.of(-2, -4, 3)));
}

private static <T> void check(String name, T expected, T actual) {
    if (!Objects.equals(expected, actual)) {
        throw new AssertionError(
            name + " expected=" + expected + " actual=" + actual
        );
    }
    System.out.println("[OK] " + name);
}
```

JUnit 5, si está disponible

```
@Test
void returnsZeroForEmptyInput() {
    assertEquals(0, sumEven(List.of()));
}

@Test
void rejectsNull() {
    assertThrows(NullPointerException.class, () -> sumEven(null));
}
```

En live coding, ejecutar cuatro casos claros desde main vale más que escribir una suite sofisticada que no llega a ejecutarse.

26. Complejidad y estructuras habituales

Operación	Complejidad típica
Recorrer n elementos	$O(n)$
Dos bucles anidados sobre n	$O(n^2)$
Ordenar	$O(n \log n)$
HashMap/HashSet get-add-contains	$O(1)$ promedio
TreeMap/TreeSet	$O(\log n)$
Buscar en lista	$O(n)$
Acceso ArrayList por índice	$O(1)$

Patrones

- Conteos: `Map<T,Integer>`.
- Duplicados/pertenencia: `Set<T>`.
- Agrupar: `Map<K,List<V>>`.
- Procesamiento FIFO: `Queue` / `ArrayDeque`.
- Pila LIFO: `Deque` / `ArrayDeque`.
- Top k: ordenar o `PriorityQueue` si el volumen lo justifica.
- Máximo/mínimo: un recorrido $O(n)$; no necesitas ordenar toda la colección.

27. Queue, Deque y PriorityQueue

```
Queue<String> queue = new ArrayDeque<>();
queue.offer("A");
queue.offer("B");
String next = queue.poll();
String peek = queue.peek();
```

```
Deque<Character> stack = new ArrayDeque<>();
stack.push('(');
char top = stack.pop();
```

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(5);
minHeap.offer(1);
int smallest = minHeap.poll();
```

- Prefiere `offer/poll/peek` cuando quieres evitar excepciones por cola vacía.
- `ArrayDeque` suele preferirse a `Stack` y `LinkedList` para pilas/colas.
- `PriorityQueue` devuelve primero el elemento con mayor prioridad, no conserva orden total al iterar.

28. Errores frecuentes en live coding

Error	Corrección mental
String con ==	Usar equals / Objects.equals
i <= size	Última posición: size - 1
Modificar List.of	Copiar a ArrayList
get de Map devuelve null	getOrDefault / containsKey
Ordenar entrada sin permiso	Ordenar una copia
División entera accidental	Convertir a double antes
Optional.get sin validar	orElse / orElseThrow
NPE por unboxing	Comprobar wrapper null
HashSet sin orden	LinkedHashSet si importa inserción
Confundir sort y sorted	sort modifica; stream.sorted crea resultado nuevo
Hora de cierre ambigua	Aclarar intervalo [inicio, fin)

Cuando te bloques

Reduce el problema a un caso pequeño, escribe la solución directa, ejecútala y verbaliza la duda exacta. No llenes el silencio inventando arquitectura.

29. Plantilla mental para resolver una función

```
public static Result solve(Input input) {  
    // 1. Validar solo lo acordado en el contrato  
  
    // 2. Crear el acumulador o estructura necesaria  
  
    // 3. Recorrer / transformar / decidir  
  
    // 4. Construir y devolver el resultado  
}
```

Antes de programar

- Repite entrada y salida.
- Pregunta por null, vacío, duplicados, orden y límites.
- Propón un ejemplo pequeño.
- Explica la estructura de datos que vas a utilizar.

Después

- Ejecuta happy path, vacío y un caso límite.
- Explica complejidad temporal y espacial.
- Indica supuestos.
- Acepta cambios de requisitos sin defender el código anterior por orgullo.

30. Ejemplo integrado: horarios de apertura

Este ejemplo muestra sintaxis relevante sin resolver una posible prueba real completa.

```
record OpeningInterval(
    DayOfWeek day,
    LocalTime start,
    LocalTime end
) {}

static boolean contains(OpeningInterval interval, LocalTime time) {
    return !time.isBefore(interval.start())
        && time.isBefore(interval.end());
}

static Map<DayOfWeek, List<OpeningInterval>> groupByDay(
    List<OpeningInterval> intervals) {
    Map<DayOfWeek, List<OpeningInterval>> result = new EnumMap<>(DayOfWeek.class);

    for (OpeningInterval interval : intervals) {
        result.computeIfAbsent(interval.day(), ignored -> new ArrayList<>())
            .add(interval);
    }
    return result;
}
```

Preguntas que debes aclarar

- ¿Puede cruzar medianoche?
- ¿Inicio incluido y cierre excluido?
- ¿Puede haber solapamientos?
- ¿Se busca una tienda, todas o una franja común?
- ¿Hay zona horaria y fechas especiales?

31. Chuleta final

```
// Lista mutable
List<T> list = new ArrayList<>();

// Unicidad
Set<T> set = new HashSet<>();

// Conteo
map.put(key, map.getOrDefault(key, 0) + 1);

// Agrupar
map.computeIfAbsent(key, ignored -> new ArrayList<>()).add(value);

// Ordenar sin mutar entrada
List<T> copy = new ArrayList<>(input);
copy.sort(comparator);

// String
Objects.equals(a, b);

// Tiempo en intervalo [start, end)
!time.isBefore(start) && time.isBefore(end);

// Vacío
return List.of();

// Fallo contractual
```

```
throw new IllegalArgumentException("Clear message");
```

No necesitas memorizar toda la biblioteca estándar. Necesitas reconocer qué herramienta encaja, consultar lo raro y dominar lo frecuente.

32. Checklist de preparación

- Puedo crear y ejecutar una clase Main sin ayuda.
- Puedo escribir bucles por índice y for-each.
- Domino List, Set y Map y sé por qué elegirlos.
- Recuerdo equals frente a ==.
- Puedo modelar datos con record, enum o clase.
- Puedo ordenar por uno o varios campos.
- Sé escribir casos desde Main y leer el error.
- Puedo explicar $O(n)$, $O(n \log n)$ y $O(n^2)$.
- Puedo utilizar LocalDate, LocalTime y DayOfWeek.
- No creo interfaces ni capas sin una razón.
- Pregunto antes de asumir reglas ambiguas.

Siguiente fase

Usaremos el entorno de práctica con enunciados progresivos. Algunas veces resolverás una función aislada; otras necesitarás records, clases, enums, interfaces o cambios de requisitos. La dificultad aumentará solo cuando puedas explicar y ejecutar la solución anterior.